

---

## **Week 5 Report: Ethical Hacking & Exploiting Vulnerabilities**

---

**By**

---

**Name: Shahbaz Iftikhar**  
**Cyber Security Intern**

---

|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>Objective.....</b>                                           | <b>3</b> |
| <b>Tools Used.....</b>                                          | <b>3</b> |
| <b>Task 1: Ethical Hacking Basics &amp; Reconnaissance.....</b> | <b>3</b> |
| Description.....                                                | 3        |
| Nmap Scan.....                                                  | 3        |
| Results.....                                                    | 3        |
| Findings.....                                                   | 4        |
| Screenshot.....                                                 | 4        |
| <b>Task 2: SQL Injection Testing.....</b>                       | <b>4</b> |
| Description.....                                                | 4        |
| Command Used.....                                               | 4        |
| Results.....                                                    | 5        |
| Findings.....                                                   | 5        |
| Security Impact.....                                            | 5        |
| Screenshot.....                                                 | 5        |
| <b>Task 3: Cross-Site Request Forgery (CSRF) Testing.....</b>   | <b>6</b> |
| Description.....                                                | 6        |
| Procedure.....                                                  | 6        |
| Findings.....                                                   | 6        |
| Security Impact.....                                            | 7        |
| Screenshots.....                                                | 7        |
| <b>Security Improvements.....</b>                               | <b>8</b> |
| SQL Injection Prevention.....                                   | 8        |
| CSRF Protection.....                                            | 9        |
| <b>Conclusion.....</b>                                          | <b>9</b> |

# Objective

The objective of this task was to learn ethical hacking techniques, perform reconnaissance on a web application, test for SQL Injection vulnerabilities, understand Cross-Site Request Forgery (CSRF), and implement security measures to protect web applications from common attacks.

## Tools Used

- Kali Linux
- OWASP Juice Shop
- Nmap
- SQLMap
- Burp Suite Community Edition
- Mozilla Firefox

## Task 1: Ethical Hacking Basics & Reconnaissance

### Description

Reconnaissance is the first phase of ethical hacking. It involves gathering information about a target system to identify potential attack surfaces.

### Nmap Scan

The following commands were used:

```
nmap localhost
```

```
nmap -sV -p 1-1000 localhost
```

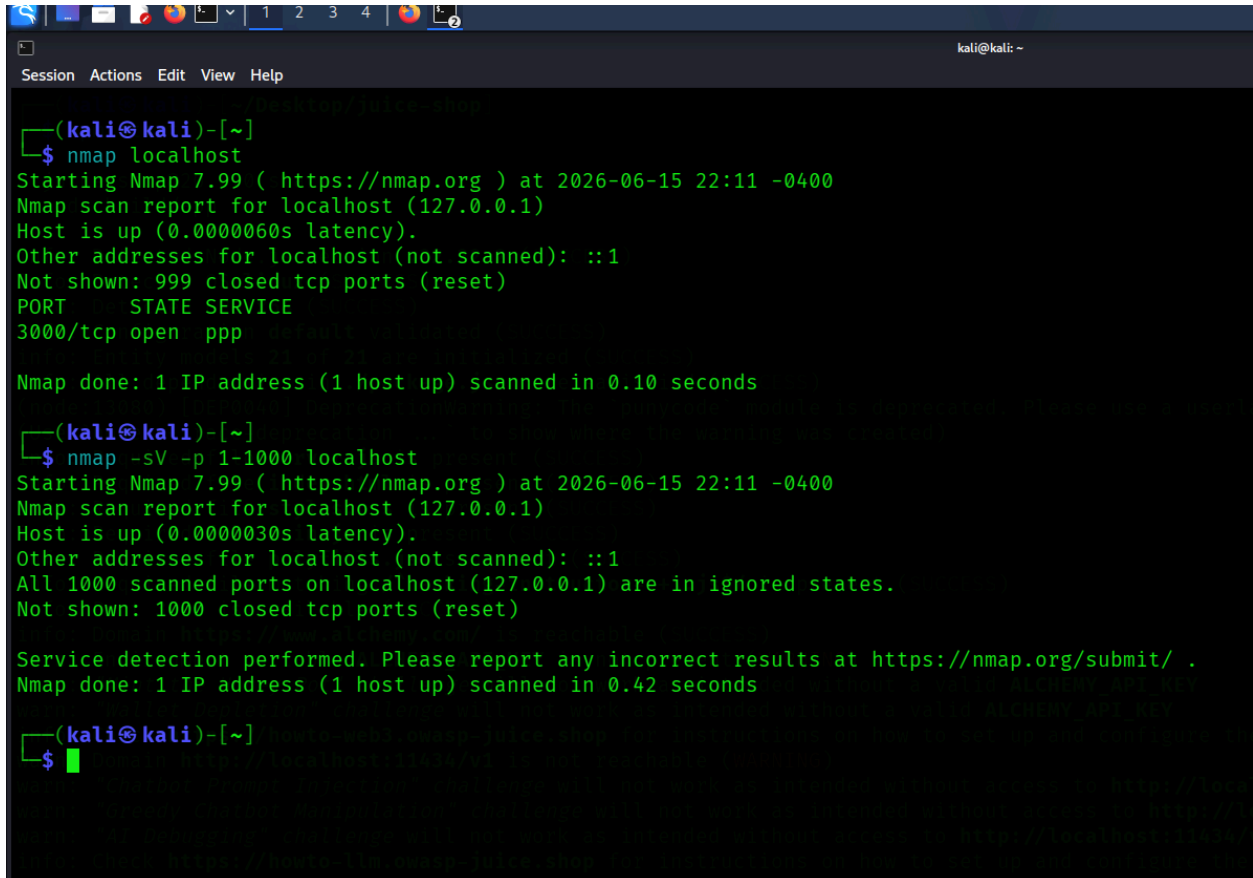
### Results

- Host was reachable.
- Port 3000/TCP was open.
- The web application was running on port 3000.
- All other scanned ports were closed.

## Findings

The scan revealed a minimal attack surface because only one service was exposed. This reduces the number of potential entry points for attackers.

## Screenshot



```
(kali㉿kali)-[~]
└─$ nmap localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-15 22:11 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000060s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
3000/tcp  open  ppp
Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds

(kali㉿kali)-[~]
└─$ nmap -sV -p 1-1000 localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-15 22:11 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000030s latency).
Other addresses for localhost (not scanned): ::1
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.42 seconds

(kali㉿kali)-[~]
└─$
```

## Task 2: SQL Injection Testing

### Description

SQL Injection testing was performed using SQLMap to determine whether the application was vulnerable to database manipulation attacks.

### Command Used

```
sqlmap -u "http://localhost:3000/login?email=test"
```

## Results

SQLMap successfully connected to the target and tested the email parameter.

The tool reported:

- Parameter was detected.
- Parameter was not dynamic.
- No SQL Injection vulnerability was found.

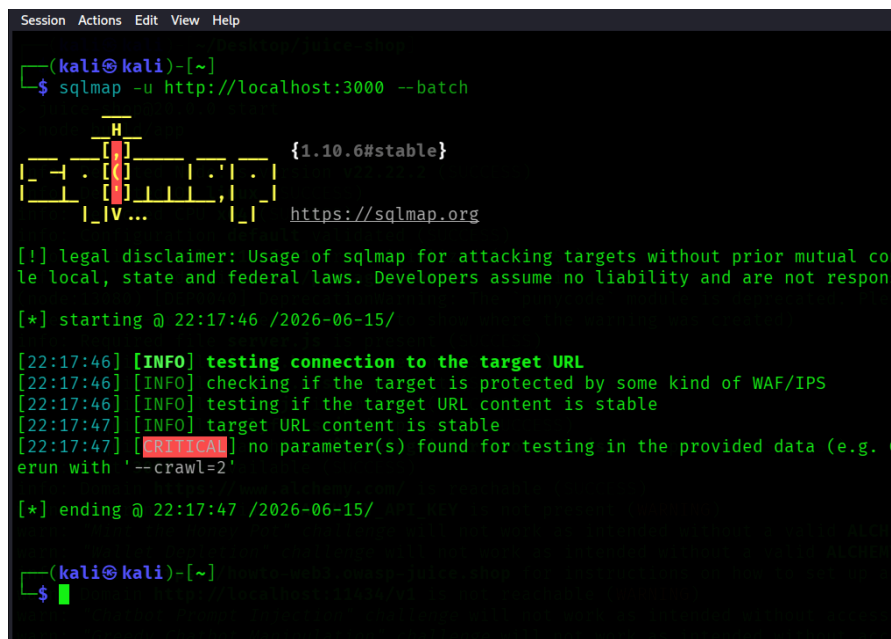
## Findings

The tested parameter did not appear injectable. The application did not expose a vulnerable SQL query through the tested endpoint.

## Security Impact

No SQL Injection vulnerability was identified during testing.

## Screenshot



```
Session Actions Edit View Help
kali@kali:~$ sqlmap -u http://localhost:3000 --batch

  ____      _
 / ___|    / \
| |  | |  / _ \
| |  | | / ___ \
| |  | |/_/   \_\
|_|  |_|_____|

{1.10.6#stable}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual co
le local, state and federal laws. Developers assume no liability and are not respon

[*] starting @ 22:17:46 /2026-06-15/

[22:17:46] [INFO] testing connection to the target URL
[22:17:46] [INFO] checking if the target is protected by some kind of WAF/IPS
[22:17:46] [INFO] testing if the target URL content is stable
[22:17:47] [INFO] target URL content is stable
[22:17:47] [CRITICAL] no parameter(s) found for testing in the provided data (e.g.
erun with '--crawl=2'

[*] ending @ 22:17:47 /2026-06-15/

kali@kali:~$
```

```
kali@kali:~$ sqlmap -u 'http://localhost:3000/login?email=test'
[+] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
[*] starting @ 22:23:05 /2020-06-15/

[22:23:06] [INFO] testing connection to the target URL
[22:23:06] [INFO] checking if the target is protected by some kind of WAF/IPS
[22:23:06] [INFO] testing if the target URL content is stable
[22:23:06] [INFO] target URL content is stable
[22:23:06] [INFO] testing if GET parameter 'email' is dynamic
[22:23:06] [WARNING] GET parameter 'email' does not appear to be dynamic
[22:23:06] [WARNING] heuristic (basic) test shows that GET parameter 'email' might not be injectable
[22:23:06] [INFO] testing for SQL injection on GET parameter 'email'
[22:23:06] [INFO] testing AND boolean-based blind - WHERE or HAVING clause
[22:23:07] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[22:23:07] [INFO] testing MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
[22:23:07] [INFO] testing PostgreSQL AND error-based - WHERE or HAVING clause
[22:23:07] [INFO] testing Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)
[22:23:07] [INFO] testing Oracle AND error-based - WHERE or HAVING clause (XMLType)
[22:23:07] [INFO] testing 'Generic inline queries'
[22:23:07] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[22:23:08] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[22:23:08] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[22:23:08] [INFO] testing 'MySQL > 5.0.12 AND time-based blind (query SLEEP)'
[22:23:08] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[22:23:08] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[22:23:08] [INFO] testing 'Oracle AND time-based blind'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] y
[22:23:25] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[22:23:25] [WARNING] GET parameter 'email' does not seem to be injectable
[22:23:25] [WARNING] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. --tamper=space2comment) and/or switch '--random-agent'

[*] ending @ 22:23:25 /2020-06-15/

kali@kali:~$
```

# Task 3: Cross-Site Request Forgery (CSRF) Testing

## Description

Burp Suite was used to intercept and analyze HTTP requests generated by the web application.

## Procedure

1. Configured Firefox to use Burp Suite proxy.
2. Enabled request interception.
3. Opened the Juice Shop application.
4. Submitted a login request.
5. Captured the request in Burp Suite.
6. Modified request parameters.
7. Forwarded the modified request to the application.
8. Replayed requests using Burp Repeater.

## Findings

- HTTP requests were successfully intercepted.
- Request parameters could be modified before forwarding.

- The testing demonstrated the importance of CSRF protection mechanisms.
- Request analysis helped identify how the application processes user input.

## Security Impact

Without proper CSRF protection, attackers may trick authenticated users into performing unintended actions.

## Screenshots

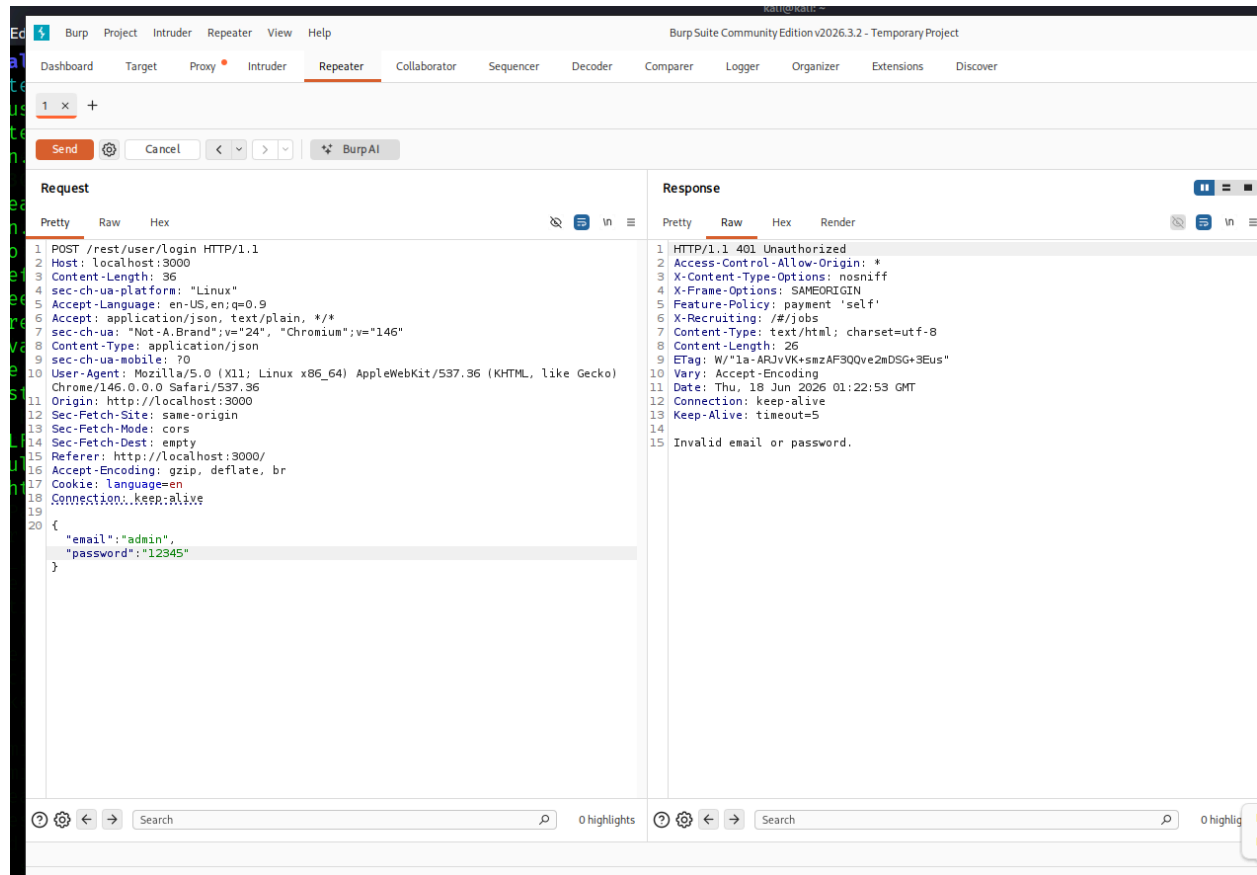
The screenshot displays the Burp Suite interface. The top menu bar includes options like Burp, Project, Intruder, Repeater, View, and Help. Below the menu, there's a toolbar with buttons for Intercept, HTTP history, WebSockets history, Match and replace, and Proxy settings. The main panel shows a list of intercepted requests with columns for Time, Type, Direction, Method, and URL. The selected request is a POST to http://localhost:3000/rest/user/login. The bottom panel provides a detailed view of this request, including headers, body, and status.

| Time                | Type | Direction | Method | URL                                                                                 | Status |
|---------------------|------|-----------|--------|-------------------------------------------------------------------------------------|--------|
| 21:20:49 17 Jun ... | HTTP | → Request | GET    | http://localhost:3000/rest/user/whoami?fields=email                                 |        |
| 21:20:49 17 Jun ... | HTTP | → Request | GET    | http://localhost:3000/rest/user/whoami                                              |        |
| 21:20:49 17 Jun ... | HTTP | → Request | POST   | http://localhost:3000/rest/user/login                                               |        |
| 21:20:49 17 Jun ... | HTTP | → Request | POST   | http://localhost:3000/rest/user/login                                               |        |
| 21:20:49 17 Jun ... | HTTP | → Request | POST   | http://localhost:3000/rest/user/login                                               |        |
| 21:20:49 17 Jun ... | HTTP | → Request | GET    | http://localhost:3000/rest/admin/application-configuration                          |        |
| 21:20:50 17 Jun ... | HTTP | → Request | GET    | http://localhost:3000/socket.io/?EIO=4&transport=websocket&sid=eZbmABJbDZGNvjHTAAAD |        |

| Request                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |     | Inspector                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|--------------------------|
| Pretty                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Raw | Request attributes       |
| <pre> 1 POST /rest/user/login HTTP/1.1 2 Host: localhost:3000 3 Content-Length: 36 4 sec-ch-ua-platform: "Linux" 5 Accept-Language: en-US,en;q=0.9 6 Accept: application/json, text/plain, */* 7 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146" 8 Content-Type: application/json 9 sec-ch-ua-mobile: ?0 10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 11 Origin: http://localhost:3000 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Dest: empty 15 Referer: http://localhost:3000/ 16 Accept-Encoding: gzip, deflate, br 17 Cookie: laravel_session= </pre> |     | Request query parameters |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |     | Request cookies          |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |     | Request headers          |

Event log All issues Memory: 128.5MB of 5.05GB



# Security Improvements

## SQL Injection Prevention

The following security practices are recommended:

- Use parameterized queries.
- Use prepared statements.
- Validate and sanitize user inputs.
- Avoid dynamic SQL query construction.

Example:

```
db.query(  
  "SELECT * FROM users WHERE email = ?",  
  [email]  
);
```



## CSRF Protection

Recommended protections include:

- CSRF Tokens
- SameSite Cookies
- Origin Validation
- Secure Session Management

Example:

```
const csrf = require('csrf');  
app.use(csrf());
```

## Conclusion

This week focused on ethical hacking fundamentals and web application security testing. Reconnaissance was performed using Nmap, SQL Injection testing was conducted using SQLMap, and HTTP request analysis was carried out using Burp Suite. No SQL Injection vulnerabilities were discovered in the tested endpoint. The activities demonstrated how attackers analyze applications and how developers can implement appropriate security controls to protect against common web attacks such as SQL Injection and CSRF.